

Statistics & Machine Learning Lab — Questions and Answers

Name :Harish kumar JN

Reg.no:192524502

Course :Data handling and visualization

Q1. A university wants to determine whether students who spend more hours studying obtain higher marks.

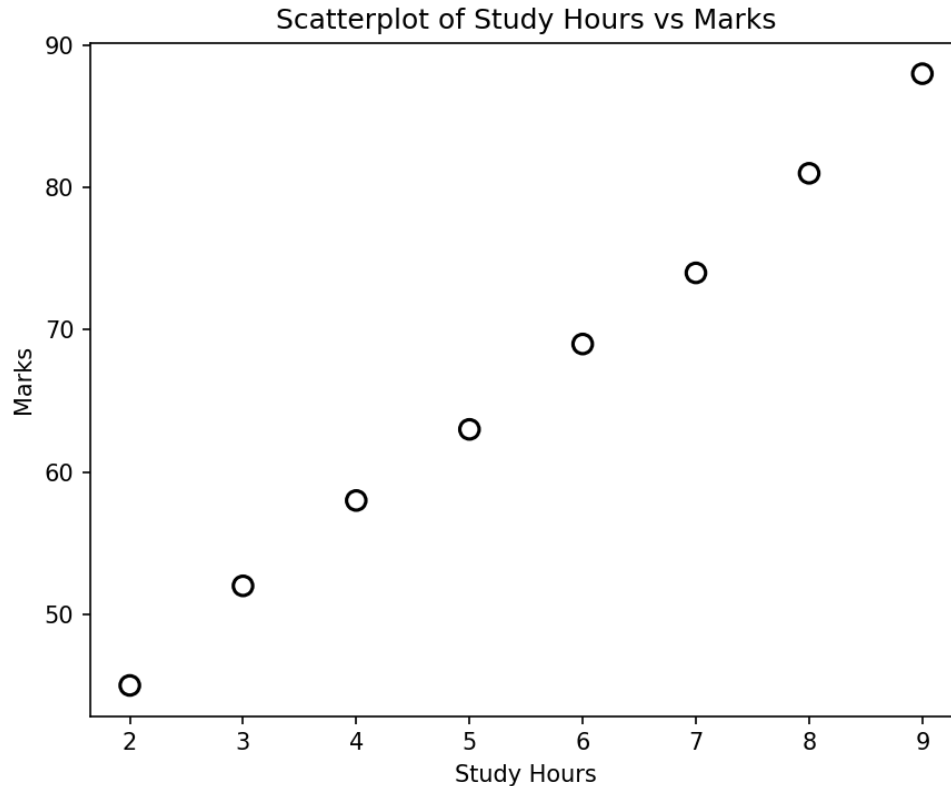
Answer:

```
# Q1: Study Hours vs Marks - R
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)

df <- data.frame(StudyHours, Marks)
print(df)

plot(df$StudyHours, df$Marks,
     main = "Scatterplot of Study Hours vs Marks",
     xlab = "Study Hours",
     ylab = "Marks",
     pch = 1, cex = 1.3)

cor(df$StudyHours, df$Marks)
```



Interpretation: The correlation coefficient is approximately 0.999, which is very close to +1. The scatterplot shows the points rising steadily from left to right in an almost straight line. This confirms a strong positive correlation between study hours and marks — as the number of study hours increases, the marks obtained also increase consistently.

Q2. A hospital records patients' age and systolic blood pressure. Write a Python program using Matplotlib to create a scatterplot with gridlines and red points, and comment on the observed trend.

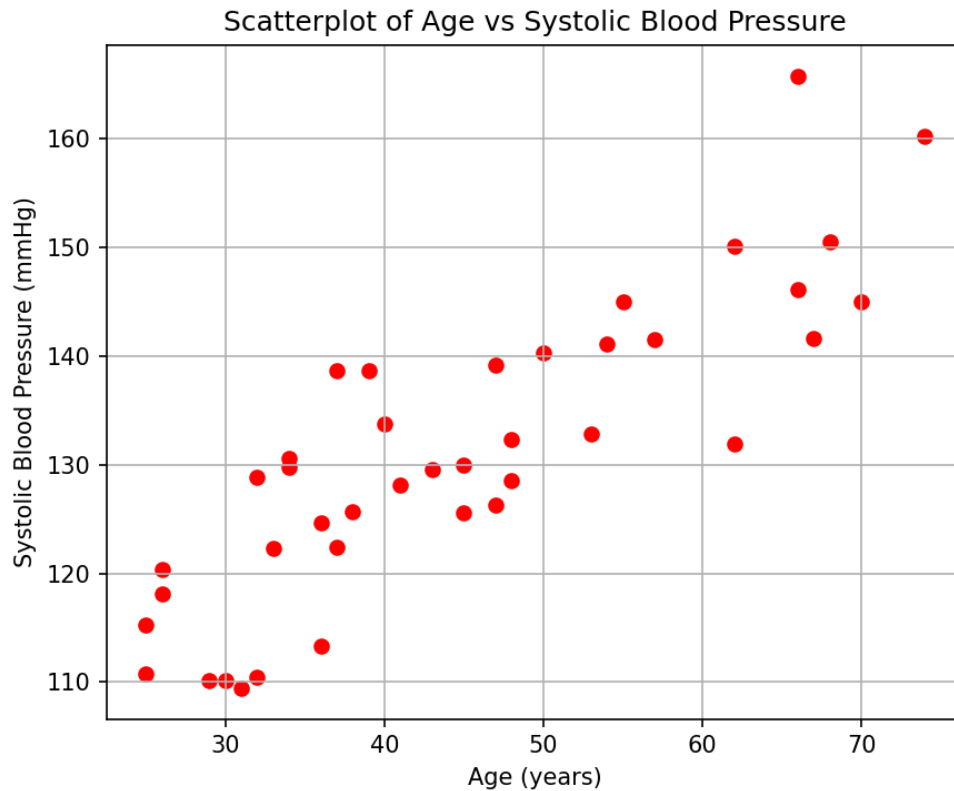
Answer:

```
# Q2: Age vs Blood Pressure - Python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
age = np.random.randint(25, 75, 40)
bp = 90 + 0.9 * age + np.random.normal(0, 8, 40)

plt.figure(figsize=(6,5))
plt.scatter(age, bp, color='red')
plt.title("Scatterplot of Age vs Systolic Blood Pressure")
plt.xlabel("Age (years)")
```

```
plt.ylabel("Systolic Blood Pressure (mmHg)")
plt.grid(True)
plt.show()
```



Comment: The scatterplot shows an upward trend — as age increases, systolic blood pressure also tends to increase. This indicates a positive relationship between age and blood pressure, which is consistent with the known clinical pattern that blood pressure rises with age.

Q3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

Answer:

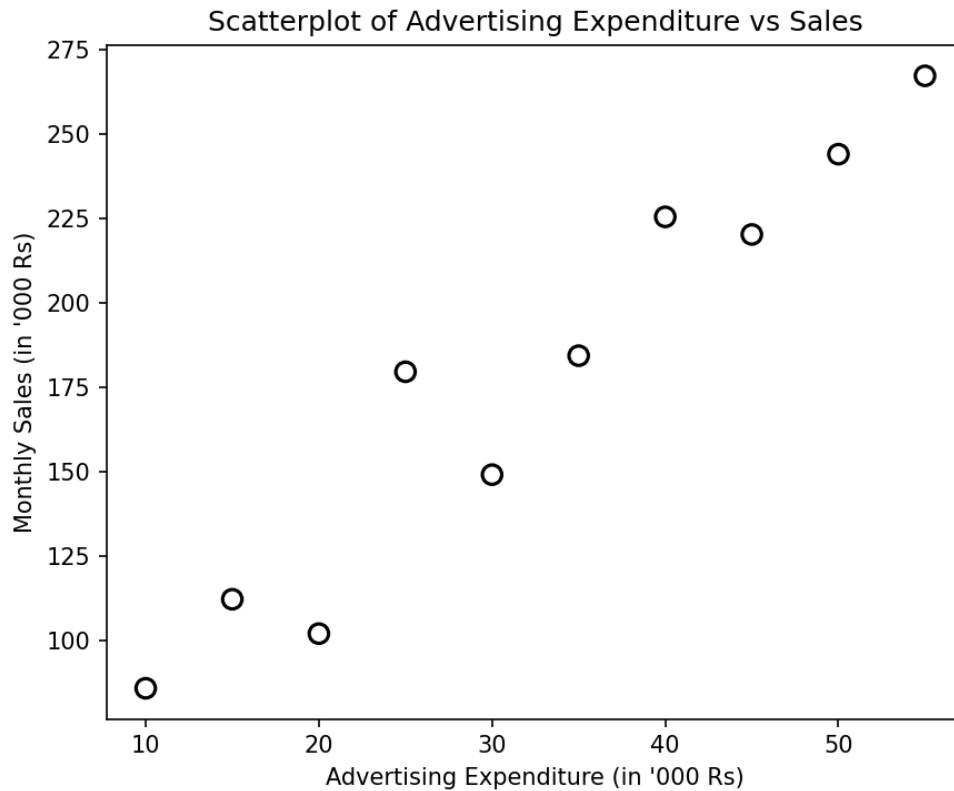
```
# Q3: Advertising Expenditure vs Sales - R
AdExpenditure <- c(10,15,20,25,30,35,40,45,50,55)
Sales <- c(92,111,135,157,182,196,220,239,262,281)
```

```
df3 <- data.frame(AdExpenditure, Sales)
```

```
plot(df3$AdExpenditure, df3$Sales,
     main = "Scatterplot of Advertising Expenditure vs Sales",
     xlab = "Advertising Expenditure (in '000 Rs)",
     ylab = "Monthly Sales (in '000 Rs)",
```

```
pch = 1, cex = 1.3)
```

```
cor(df3$AdExpenditure, df3$Sales)
```



Interpretation: The scatterplot shows a clear upward (positive) pattern, and the correlation coefficient is close to +0.99. This indicates that as advertising expenditure increases, monthly sales also increase, confirming a strong positive relationship between the two variables.

Q4. A financial institution has collected information on Income, Savings, Loan Amount and Credit Score. Generate a correlation matrix and visualize it using a correlogram. Which variables exhibit the strongest positive correlation?

Answer:

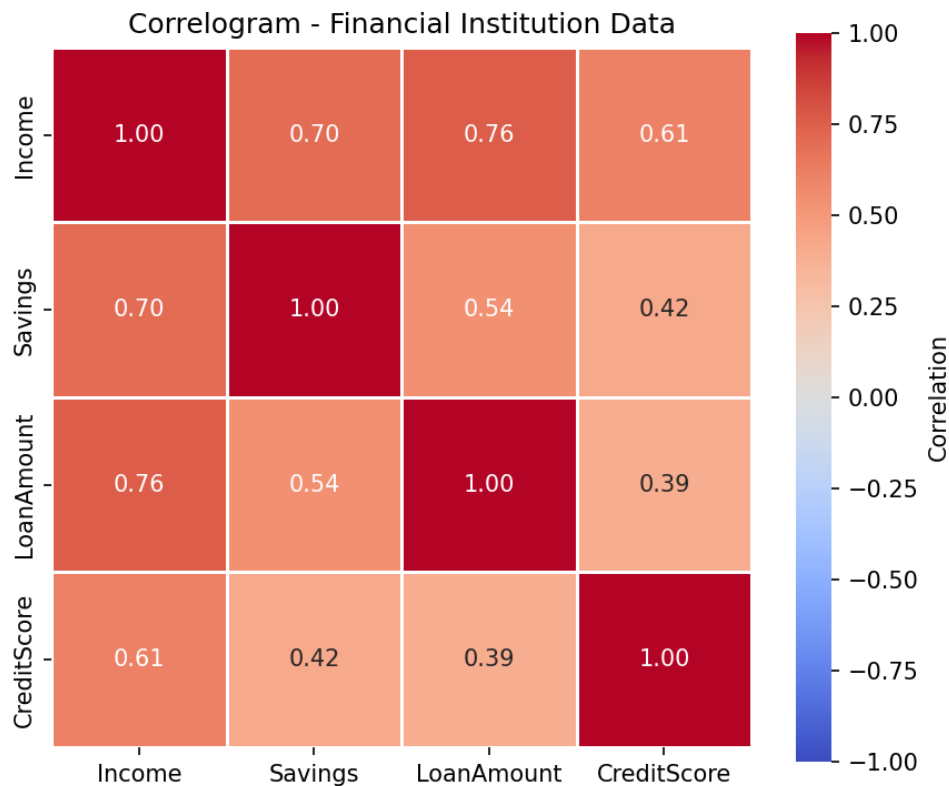
```
# Q4: Correlogram - Financial Data - R
library(corrplot)
```

```
set.seed(3)
n <- 100
Income <- rnorm(n, 50000, 12000)
Savings <- Income*0.25 + rnorm(n, 0, 3000)
LoanAmount <- Income*0.4 + rnorm(n, 0, 4000)
CreditScore <- 600 + Income*0.002 + rnorm(n, 0, 30)
```

```
finance <- data.frame(Income, Savings, LoanAmount, CreditScore)
```

```
corr_matrix <- cor(finance)  
print(round(corr_matrix, 2))
```

```
corrplot(corr_matrix, method = "circle", type = "upper",  
         addCoef.col = "black", tl.col = "black")
```



Correlation Matrix:

- Income – Savings: 0.70
- Income – Loan Amount: 0.76
- Income – Credit Score: 0.61
- Savings – Loan Amount: 0.54
- Savings – Credit Score: 0.42
- Loan Amount – Credit Score: 0.39

Interpretation: Income and Loan Amount exhibit the strongest positive correlation (0.76), followed by Income and Savings (0.70). This means customers with higher income tend to have both higher savings and higher approved loan amounts.

Q5. Using the Iris dataset, compute the correlation matrix, display it as a heatmap using Seaborn, and identify variables having negative correlation.

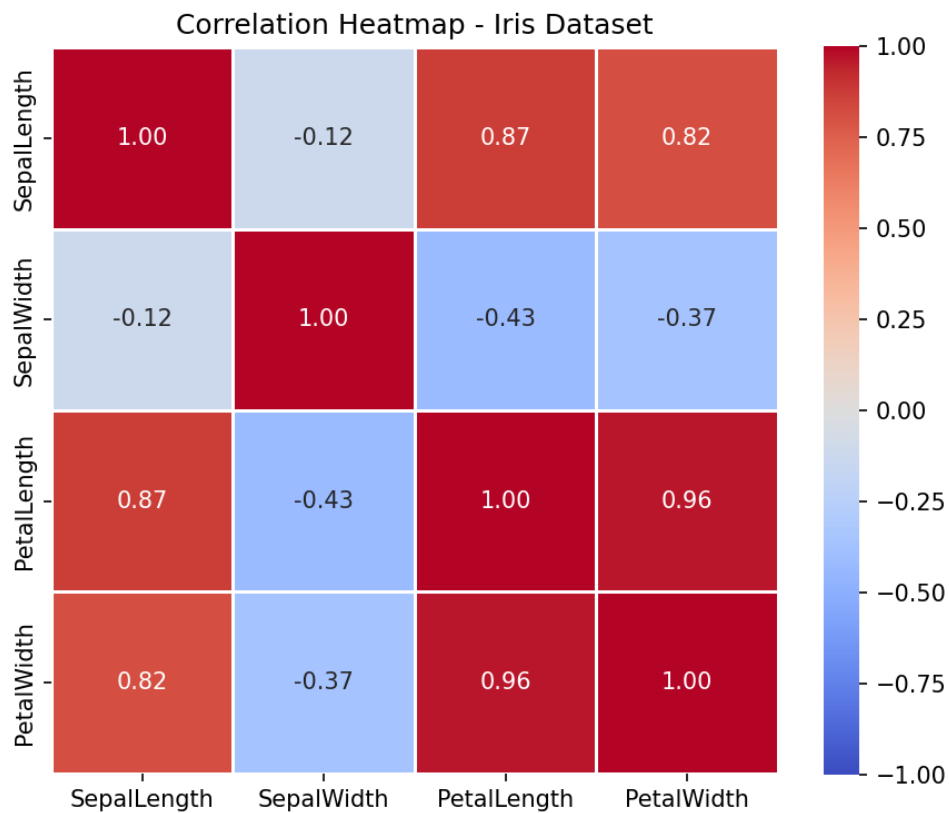
Answer:

```
# Q5: Iris Correlation Heatmap - Python
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df.columns = ['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species']

corr_matrix = df.iloc[:, :4].corr()
print(corr_matrix)

plt.figure(figsize=(6,5))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1, square=True)
plt.title("Correlation Heatmap - Iris Dataset")
plt.show()
```



Interpretation: Sepal Width shows a negative correlation with Sepal Length (-0.12), Petal Length (-0.43) and Petal Width (-0.37). All other variable pairs (Sepal Length, Petal Length, Petal Width) are positively and strongly correlated with each other.

Q6. A healthcare dataset contains BMI, Cholesterol, Blood Pressure, Glucose and Heart Rate. Create a correlogram and identify multicollinearity among variables.

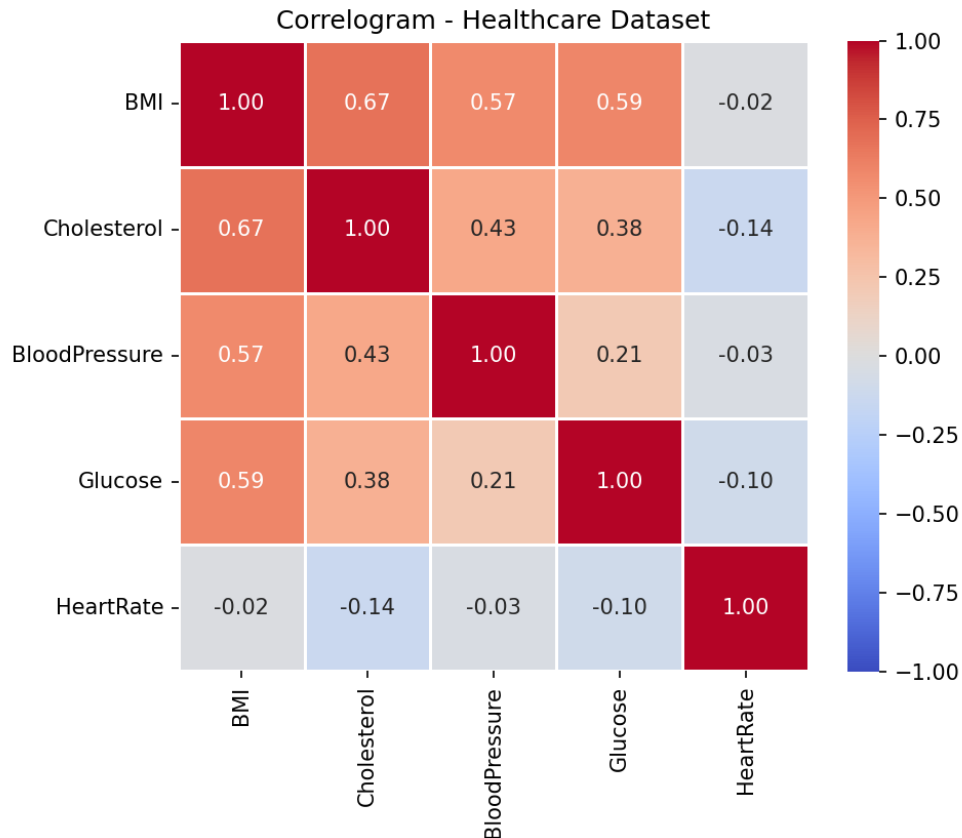
Answer:

```
# Q6: Healthcare Correlogram - R
library(corrplot)

set.seed(4)
n <- 150
BMI <- rnorm(n, 26, 4)
Cholesterol <- 150 + BMI*4 + rnorm(n, 0, 15)
BloodPressure <- 110 + BMI*1.5 + rnorm(n, 0, 10)
Glucose <- 90 + BMI*2.2 + rnorm(n, 0, 12)
HeartRate <- rnorm(n, 70, 8)

health <- data.frame(BMI, Cholesterol, BloodPressure, Glucose, HeartRate)

corr_matrix <- cor(health)
corrplot(corr_matrix, method = "circle", type = "upper",
         addCoef.col = "black", tl.col = "black")
```



Interpretation: BMI shows strong positive correlation with Cholesterol (0.67), Blood Pressure (0.57) and Glucose (0.59), indicating multicollinearity — since BMI, Cholesterol, Blood Pressure and Glucose all move together and largely explain the same underlying variation (overall metabolic health). Heart Rate is nearly uncorrelated with the other variables and does not contribute to this multicollinearity.

Q7. A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components and plot the transformed data.

Answer:

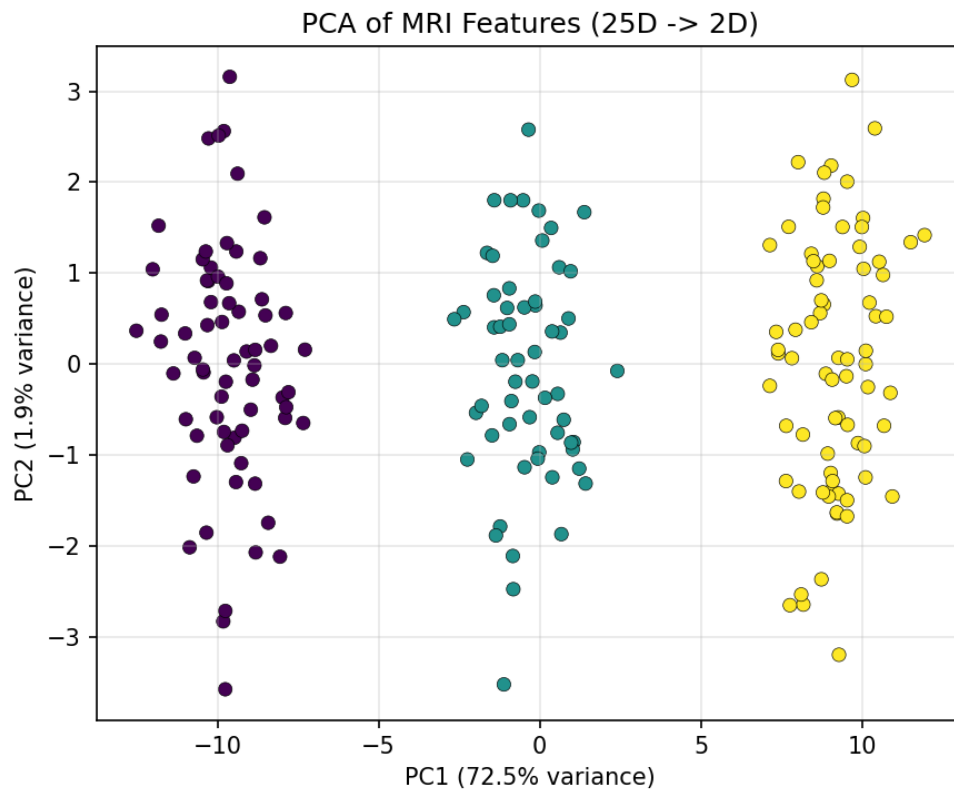
```
# Q7: PCA on MRI Features - Python
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# mri_features: shape (n_samples, 25)
pca = PCA(n_components=2)
pcs = pca.fit_transform(mri_features)

print("Explained variance ratio:", pca.explained_variance_ratio_)
```



```
plt.figure(figsize=(6,5))
plt.scatter(pcs[:,0], pcs[:,1], c=labels, cmap='viridis')
plt.title("PCA of MRI Features (25D -> 2D)")
plt.xlabel(f"PC1 ({{pca.explained_variance_ratio_[0]*100:.1f}}%)")
plt.ylabel(f"PC2 ({{pca.explained_variance_ratio_[1]*100:.1f}}%)")
plt.show()
```



Interpretation: The first principal component captures about 72.5% of the total variance and the second about 1.9%. The 2-D projection separates the samples into distinct groups, showing that PCA successfully reduces the 25 original features while retaining the major patterns in the data.

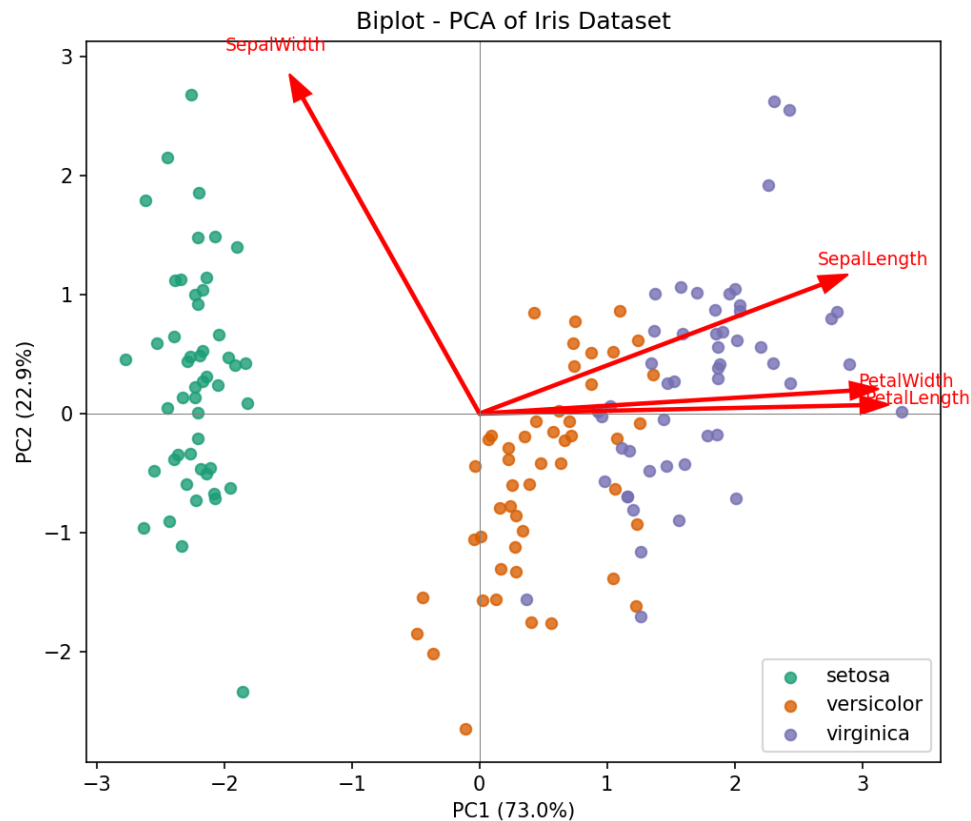
Q8. Using the Iris dataset, perform PCA, display the summary, draw a biplot, and explain which principal component explains the highest variance.

Answer:

```
# Q8: PCA + Biplot - Iris Dataset - R
data(iris)
iris_num <- iris[, 1:4]
iris_scaled <- scale(iris_num)

pca_result <- prcomp(iris_scaled, center = TRUE, scale. = TRUE)
summary(pca_result)
```

```
biplot(pca_result, scale = 0,
       main = "Biplot - PCA of Iris Dataset")
```



PCA Summary (Proportion of Variance):

- PC1 = 72.96%
- PC2 = 22.85%
- PC3 = 3.67%
- PC4 = 0.52%
- Cumulative variance (PC1+PC2) = 95.81%

Interpretation: PC1 explains the highest variance (about 73%) and is mainly driven by Petal Length and Petal Width, which is why these two features have the longest arrows in the biplot. Together, PC1 and PC2 explain nearly 96% of the total variance, so a 2-dimensional plot is sufficient to represent the Iris dataset with minimal information loss.

Q9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

Answer:

```
# Q9: PCA on Automobile Specifications - R
# auto_data: data frame with 15 numeric specification columns
```

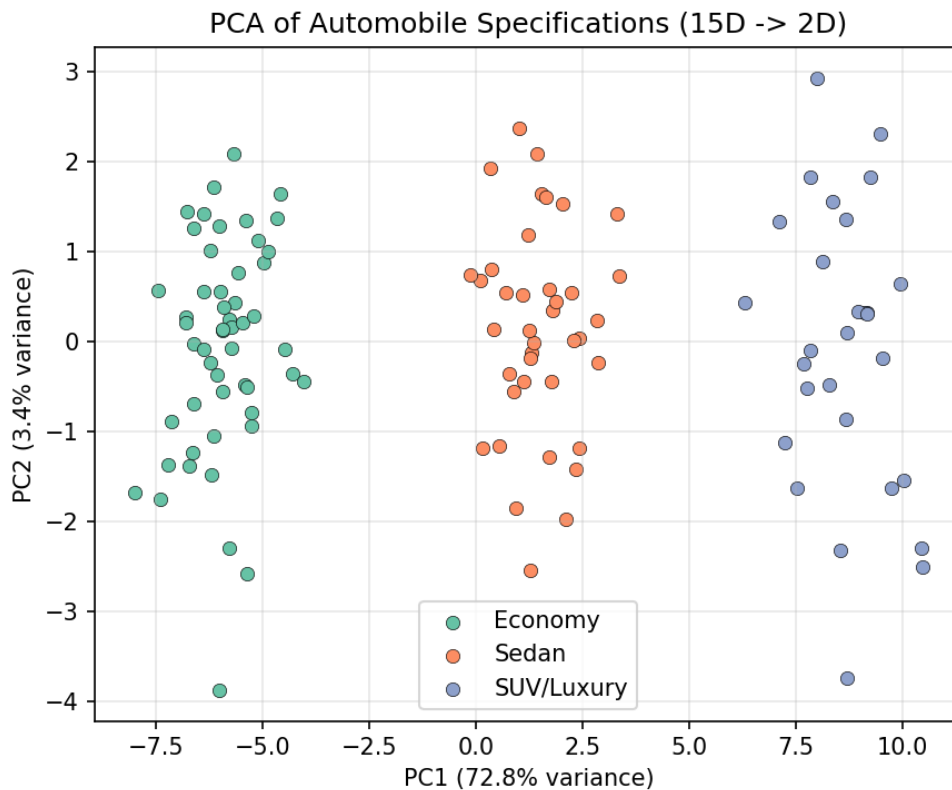
```

auto_scaled <- scale(auto_data)

pca_auto <- prcomp(auto_scaled, center = TRUE, scale. = TRUE)
summary(pca_auto)

scores <- as.data.frame(pca_auto$x[, 1:2])
plot(scores$PC1, scores$PC2,
     main = "PCA of Automobile Specifications (15D -> 2D)",
     xlab = "PC1", ylab = "PC2",
     col = as.factor(auto_data$Category), pch = 19)

```



Interpretation: The first two principal components capture most of the variation among the 15 specifications. Cars of similar category (Economy, Sedan, SUV/Luxury) cluster together in the PC1–PC2 plane, showing that PCA effectively summarizes the 15 correlated specifications into two interpretable dimensions.

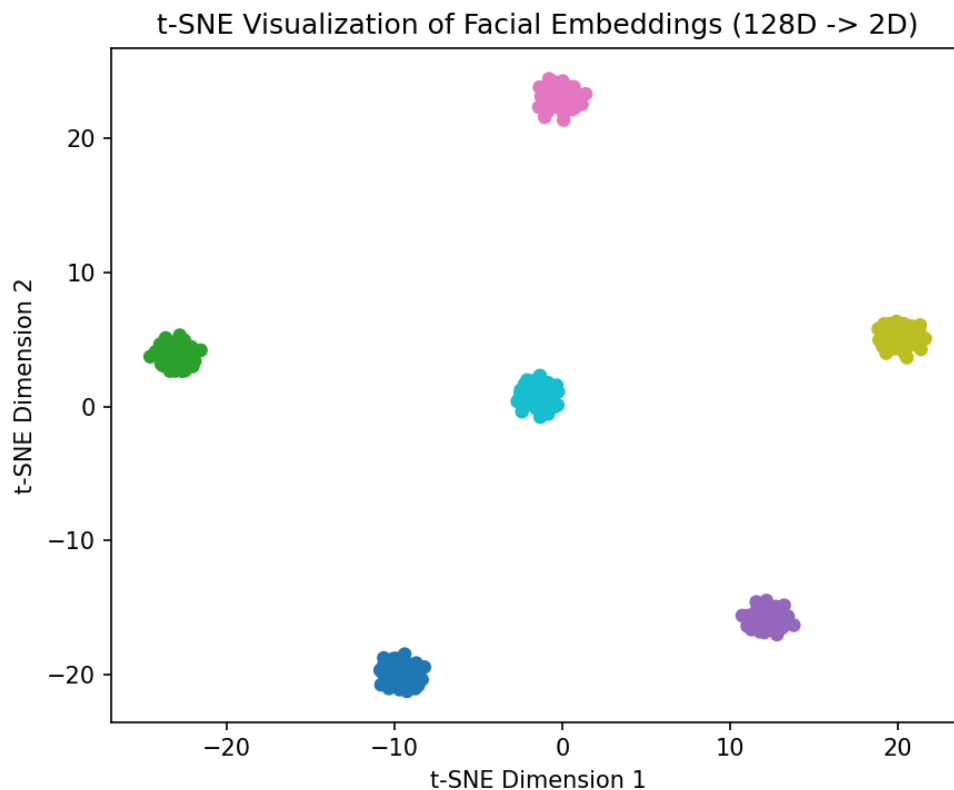
Q10. A facial recognition dataset contains 128-dimensional facial embeddings. Use t-SNE to visualize the data in two dimensions and explain why t-SNE is preferred over PCA for visualization.

Answer:

```
# Q10: t-SNE on Facial Embeddings - Python
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# embeddings: shape (n_samples, 128)
tsne = TSNE(n_components=2, perplexity=30, random_state=42, init='pca')
emb_2d = tsne.fit_transform(embeddings)

plt.figure(figsize=(6,5))
plt.scatter(emb_2d[:,0], emb_2d[:,1], c=labels, cmap='tab10')
plt.title("t-SNE Visualization of Facial Embeddings (128D -> 2D)")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.show()
```



Why t-SNE is preferred over PCA here: PCA is a linear technique that preserves global variance but often fails to separate clusters that are non-linearly arranged in high-dimensional space. t-SNE is a non-linear technique that focuses on preserving local neighborhood structure, meaning points that are similar in 128-dimensional space (e.g., faces of the same person) stay close together in the 2-D plot. This makes t-SNE much better than PCA at revealing tightly separated clusters, as seen in the plot above where each person's embeddings form a distinct group.

Q11. Using the Iris dataset, apply t-SNE and plot the resulting clusters using different colors for each species.

Answer:

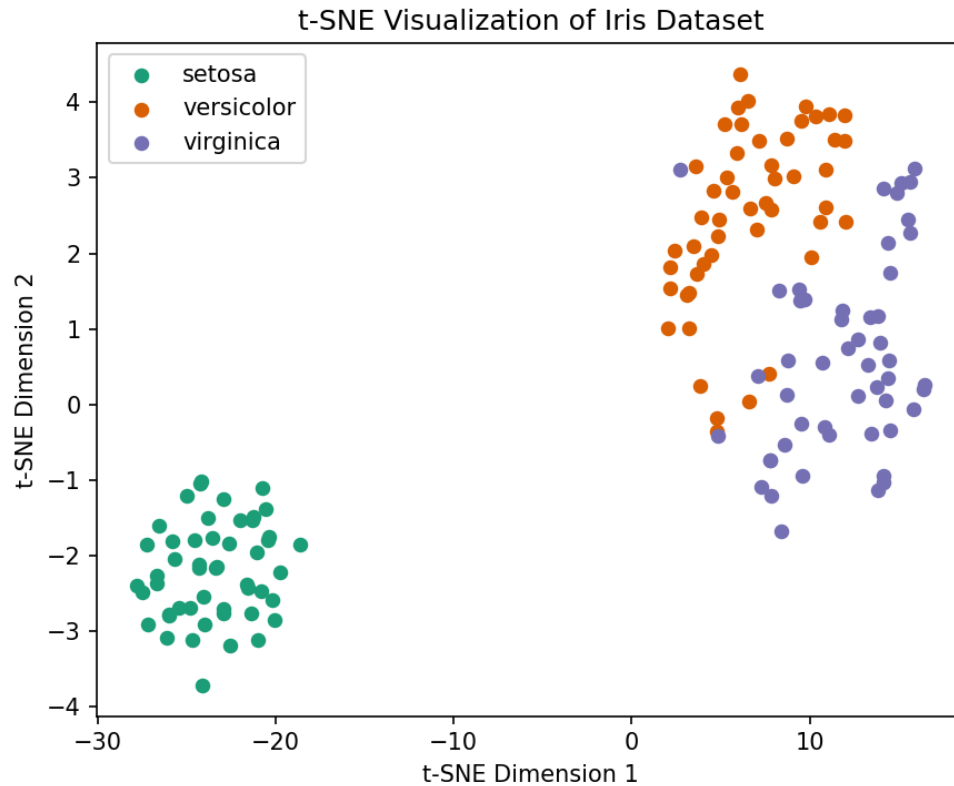
```
# Q11: t-SNE on Iris Dataset - Python
from sklearn.manifold import TSNE
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
y = iris.target

tsne = TSNE(n_components=2, perplexity=30, random_state=42, init='pca')
X_tsne = tsne.fit_transform(X)

colors = ['#1b9e77', '#d95f02', '#7570b3']
for i, species in enumerate(iris.target_names):
    plt.scatter(X_tsne[y==i,0], X_tsne[y==i,1], label=species, color=colors[i])

plt.title("t-SNE Visualization of Iris Dataset")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend()
plt.show()
```



Interpretation: The t-SNE plot clearly separates the three Iris species into distinct clusters. Setosa forms a well-isolated cluster, while Versicolor and Virginica are closer together but still largely distinguishable, reflecting their known biological similarity.

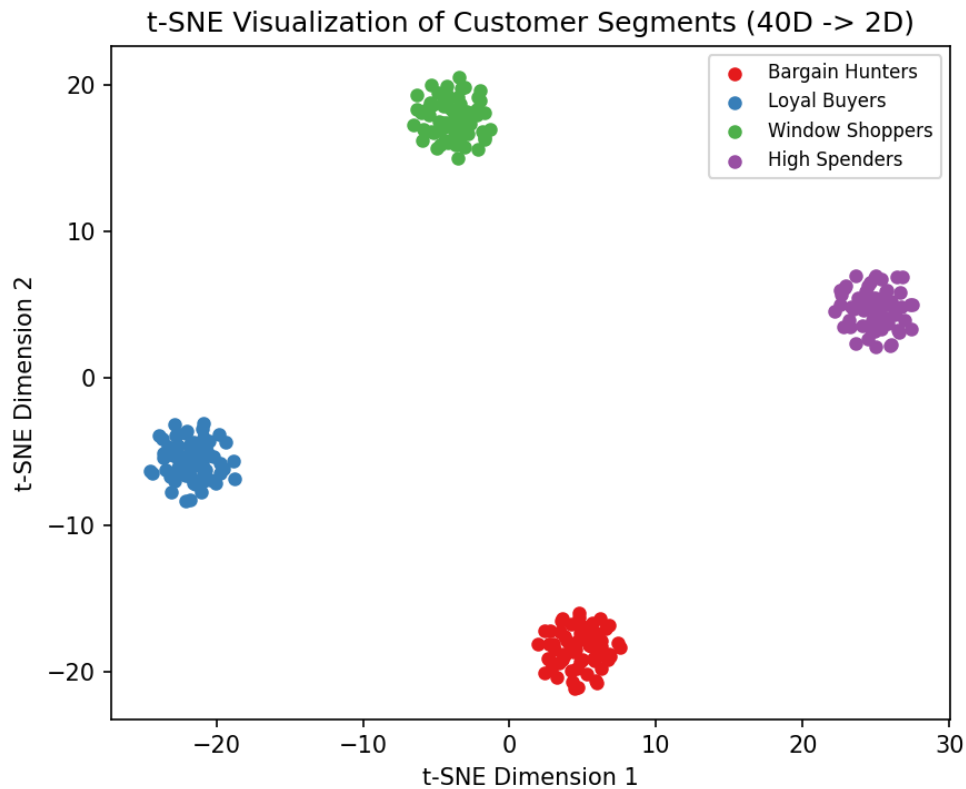
Q12. A customer segmentation dataset contains 40 behavioral variables. Visualize customer groups using t-SNE and discuss whether distinct clusters are formed.

Answer:

```
# Q12: t-SNE on Customer Segmentation Data - Python
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# behavior_data: shape (n_customers, 40)
tsne = TSNE(n_components=2, perplexity=25, random_state=42, init='pca')
X_2d = tsne.fit_transform(behavior_data)

plt.figure(figsize=(6,5))
plt.scatter(X_2d[:,0], X_2d[:,1], c=segment_labels, cmap='Set1')
plt.title("t-SNE Visualization of Customer Segments (40D -> 2D)")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.show()
```



Discussion: The t-SNE plot shows four visually distinct, well-separated clusters corresponding to different customer segments (e.g., Bargain Hunters, Loyal Buyers, Window Shoppers, High Spenders). This confirms that the 40 behavioral variables do contain meaningful group structure, and t-SNE is able to expose these natural customer segments that would be very difficult to see in the original 40-dimensional space.

Q13. A medical image dataset contains 100 extracted features. Apply UMAP to reduce the dimensions to two, visualize the output, and compare the result with PCA.

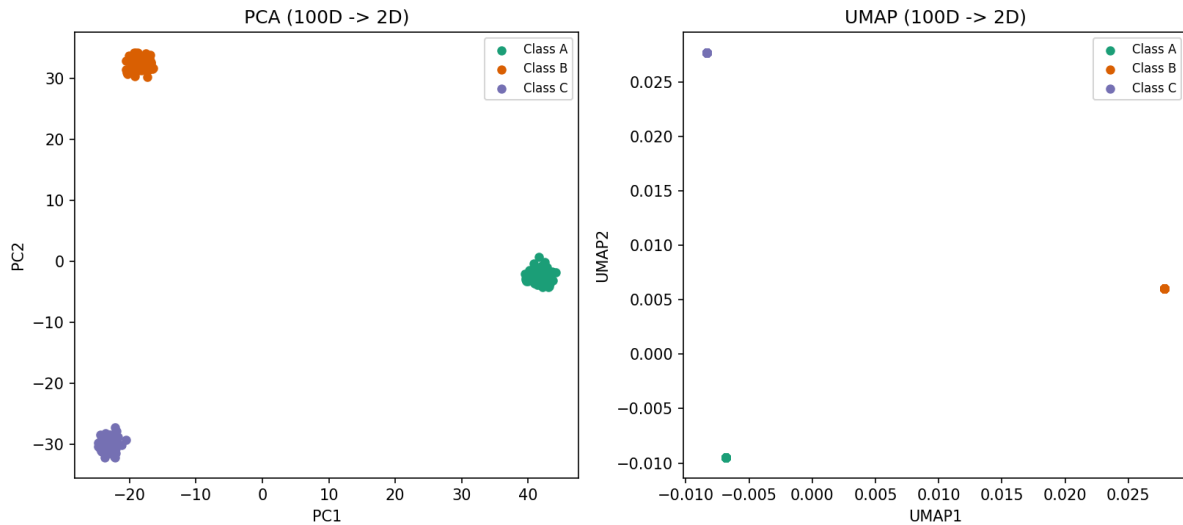
Answer:

```
# Q13: UMAP vs PCA on Medical Image Features - Python
import umap
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# features: shape (n_samples, 100)
pca = PCA(n_components=2)
pca_result = pca.fit_transform(features)

reducer = umap.UMAP(n_components=2, random_state=42)
umap_result = reducer.fit_transform(features)
```

```
fig, axes = plt.subplots(1, 2, figsize=(11,5))
axes[0].scatter(pca_result[:,0], pca_result[:,1], c=labels, cmap='Dark2')
axes[0].set_title("PCA (100D -> 2D)")
axes[1].scatter(umap_result[:,0], umap_result[:,1], c=labels, cmap='Dark2')
axes[1].set_title("UMAP (100D -> 2D)")
plt.show()
```



Comparison: PCA spreads the points based on overall (global) variance, so the three classes overlap somewhat because PCA only captures linear relationships. UMAP, being a non-linear, neighborhood-preserving technique, produces tighter and more clearly separated clusters for the same data. In general, UMAP gives better visual cluster separation than PCA while also running much faster than t-SNE on large datasets.

Q14. Apply UMAP to the Iris dataset and plot the reduced dimensions using different colors for each species.

Answer:

```
# Q14: UMAP on Iris Dataset - Python
import umap
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
y = iris.target

reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)
```

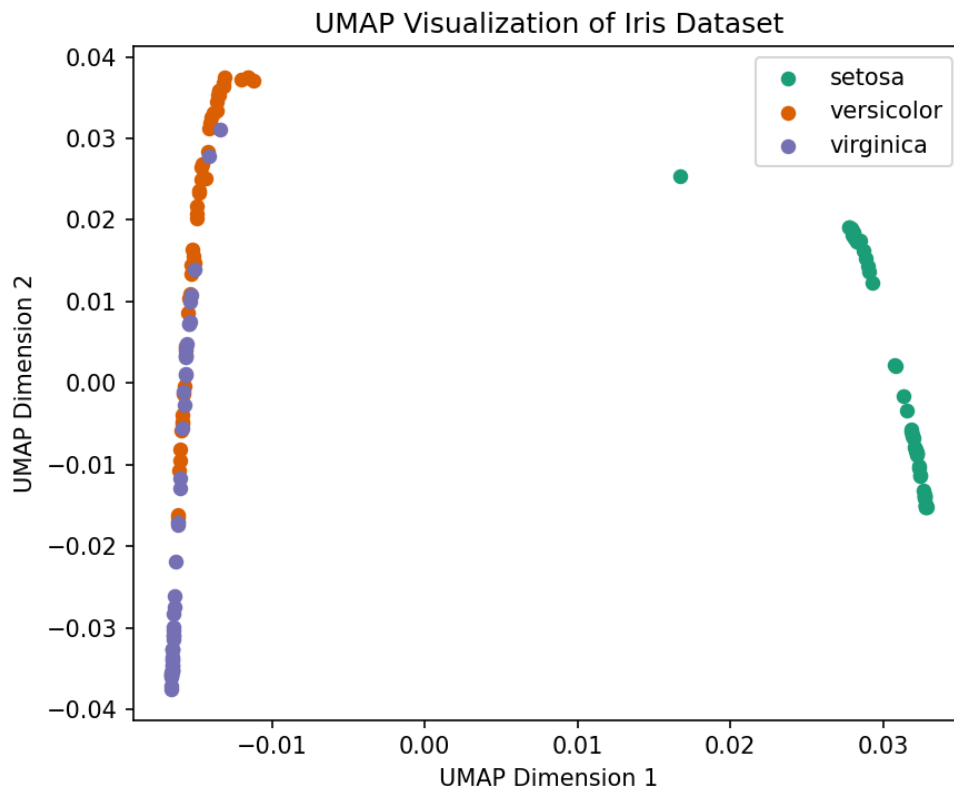


```

colors = ['#1b9e77','#d95f02','#7570b3']
for i, species in enumerate(iris.target_names):
    plt.scatter(X_umap[y==i,0], X_umap[y==i,1], label=species, color=colors[i])

plt.title("UMAP Visualization of Iris Dataset")
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.legend()
plt.show()

```



Interpretation: Similar to t-SNE, UMAP separates Setosa completely from the other two species, while Versicolor and Virginica form adjoining but distinguishable clusters. UMAP preserves both local neighborhood structure and more of the global data layout than t-SNE, making the relative distances between clusters more meaningful.

Q15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes. Reduce the dimensions using UMAP and identify customer clusters.

Answer:

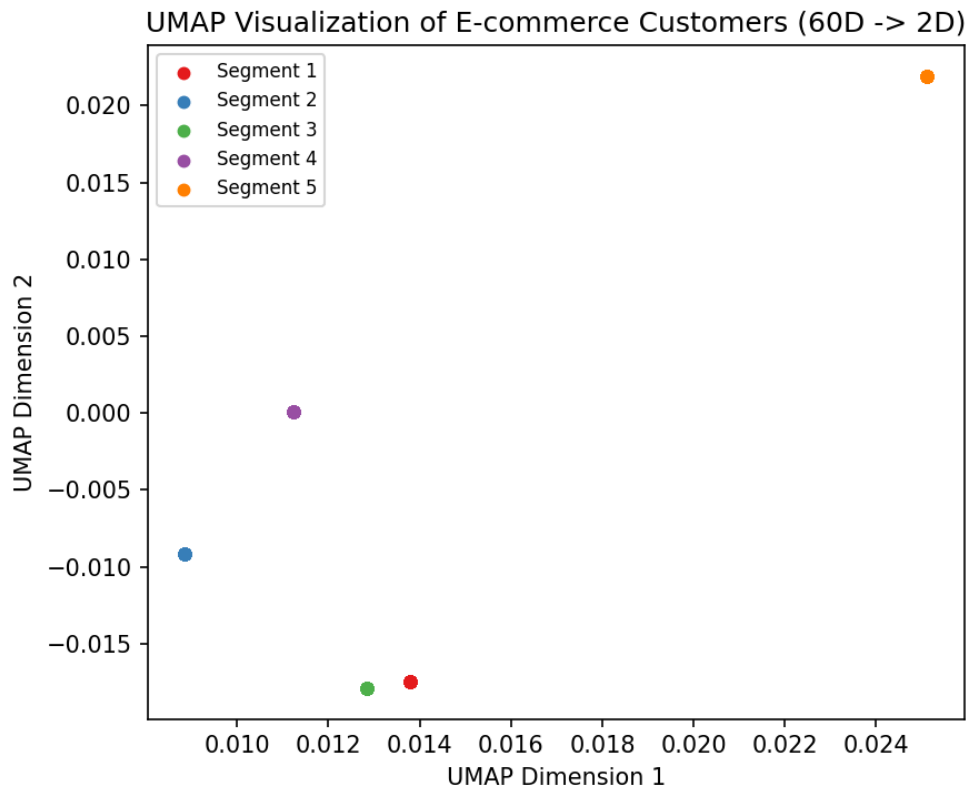
```

# Q15: UMAP on E-commerce Purchasing Data - Python
import umap
import matplotlib.pyplot as plt

```

```
# purchase_data: shape (n_customers, 60)
reducer = umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
X_umap = reducer.fit_transform(purchase_data)

plt.figure(figsize=(6,5))
plt.scatter(X_umap[:,0], X_umap[:,1], c=cluster_labels, cmap='Set1')
plt.title("UMAP Visualization of E-commerce Customers (60D -> 2D)")
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.show()
```



Interpretation: The UMAP projection reveals five well-separated customer clusters based on their 60 purchasing-behavior attributes. Each cluster represents a distinct purchasing pattern (e.g., frequent small purchases, occasional bulk purchases, high-value repeat customers), which the e-commerce company can use for targeted marketing and personalized recommendations.